

explicaciones técnicas adicionales.

Carlos Andrés Zuluaga Acuña

Luis Fernando Pérez Chala

Robert Andrés Osorio Patiño

Docente: Jesús Ariel González Bonilla

Estructura de datos

Programa: ING de Sistemas

Corporación Universitaria Del Huila Corhuila

Año 2025

Primera Parte: Código con estructuras básicas

Introducción

En esta primera parte del proyecto final desarrollamos un programa en Java que permite simular pedidos en un restaurante.

Está hecho con programación estructurada, es decir, sin usar clases ni objetos, solo con estructuras básicas como condiciones, ciclos y arreglos.

¿Qué hace el programa?

El usuario puede ver un menú de restaurante con opciones como hamburguesas, pizza, ensalada o bebidas.

Luego elige los productos que quiere, ingresa cuántas unidades desea de cada uno, y al final el sistema le muestra un resumen del pedido con el total a pagar. Si el total supera los \$50.000, el sistema aplica automáticamente un descuento del 10%.

¿Cómo está hecho?

- **Condicionales (if, else if, else)**
Para validar que las cantidades sean válidas y aplicar el descuento.
- **Un switch**
Que controla el menú principal. Cada opción hace algo diferente.
- **Un ciclo for**
Que recorre los productos y calcula subtotales.
- **Un do-while**
Para que el menú se repita hasta que el usuario elija salir.
- **Vectores (arreglos)**
Para guardar los nombres de los platos, los precios y cuántas unidades pidió el usuario de cada uno

Variables que se usan

- **platos:** guarda los nombres de cada comida.
- **precios:** guarda cuánto vale cada plato.
- **cantidades:** guarda cuántas unidades pidió el usuario.
- **opcion:** controla lo que elige el usuario en el menú.
- **total:** suma de todo lo que pidió.

Cosas extra que hace

- Si el usuario pone una cantidad negativa, muestra un mensaje de error.
- Si pone cero, no se agrega nada.
- Si el total supera \$50.000, se aplica un descuento del 10% automáticamente.

Conclusión

Este programa es simple pero completo. Usa todas las estructuras que vimos en clase y cumple con el objetivo: simular pedidos como si estuviéramos en un restaurante. Aunque está hecho con programación estructurada, funciona bien, es fácil de entender y da una buena base para luego mejorar el proyecto con POO.

Segunda Parte: Ejercicio de Programación Orientada a Objetos (POO)

Introducción

En esta segunda parte del proyecto hicimos una versión mejor organizada del sistema de pedidos, pero esta vez usando Programación Orientada a Objetos (POO) en Java.

¿Qué hace el programa?

El usuario puede ver las categorías del menú (como hamburguesas o bebidas), elegir un producto dentro de cada categoría, y agregarlo a su pedido. Después puede revisar lo que seleccionó y ver el total a pagar. Si el total supera los \$50.000, el sistema aplica un 10% de descuento.

¿Cómo está hecho?

El código está dividido en varias clases:

- **Producto:** guarda nombre, descripción y precio de cada producto.
- **Categoria:** agrupa productos del mismo tipo.
- **RestauranteApp:** es la clase principal que muestra el menú y controla todo.

Se usaron métodos para mostrar productos, manejar el carrito y calcular el total. También se aplicaron los principios de POO: encapsulamiento (private + getters), objetos (new Producto(...)), e interacción entre clases (Categoria contiene productos, App usa todo).

Variables importantes

- **ArrayList<Producto>**: para guardar el pedido del usuario.
- **Scanner**: para leer las opciones del menú.
- **opcion**: para manejar la navegación del menú.
- **total**: suma de todos los productos seleccionados.

Cosas destacadas

- Se separó todo por clases para que el código sea más ordenado.
- Se aplicaron bien los conceptos de POO como instanciación y encapsulamiento.
- Se creó un menú más limpio y modular, donde se pueden agregar más categorías o productos fácilmente si se quisiera ampliar.

Conclusión

Esta parte del proyecto fue útil para entender cómo aplicar POO en un caso real. Organizar el código en clases lo hizo mucho más claro y escalable. Además, se aplicaron varios conceptos clave del curso, no solo de POO, sino también estructuras como arreglos dinámicos (ArrayList) y condicionales. Fue una mejora clara respecto a la versión estructurada.